NYC 311 Service Requests Lakehouse

Phase 1 Project Proposal

Team: Talmeez ur Rehman (24L-2576) and Mahad Jawad (24L-2504)

GitHub repository: https://github.com/talmeez450/nyc311-medallion-lakehouse

---

1. Introduction

This project builds a batch data pipeline for New York City 311 service request data using Apache Spark and Delta Lake on Databricks. Raw data is loaded into a Bronze layer, cleansed and de-duplicated in a Silver layer, and modeled as a star schema in a Gold layer that feeds a dashboard. The pipeline has two load paths: a one-time Full Load of an eight-month baseline, and a daily Incremental Load from the Socrata API that keeps the tables current. Because 311 tickets change status over time, updates are applied with Delta Lake MERGE INTO. Free-text fields that may contain personal information are removed before data reaches the Silver layer.

2. Domain and Source Identification

2.1 Domain

The domain is civic technology and urban incident management. The 311 system is New York City's non-emergency service line. Residents report problems such as noise, rodents, illegal parking, or missed garbage collection, and the request is routed to the responsible city agency (NYPD, DSNY, DOT, HPD, and others) until it is closed.

2.2 Data Source

Item	Detail
Provider	NYC Open Data portal (Socrata)
Dataset ID	erm2-nwe9
Full Load access	Bulk CSV export, filtered by created_date
Incremental access	Socrata SODA REST API (JSON), filtered with SoQL $where
Primary key	unique_key
Fields that change over time	status, closed_date, resolution_action_updated_date, resolution_description

2.3 Full Load

February 1, 2026 up to a cut-off timestamp T0 (October 4, 2026, 00:00), which is the moment the export is taken. T0 is saved in a pipeline control table and serves as the starting watermark for incremental loading.

2.4 Incremental Load

A parameterized notebook calls the Socrata REST API once per day and retrieves every record that was created or modified in the look-back window:

```
$where = created_date >= '{window_start}'
     OR (resolution_action_updated_date >= '{window_start}'
         AND resolution_action_updated_date <= '{run_time}')

window_start = LEAST(last_watermark, current_date - 7 days)
```

Requests are paginated with $limit and $offset, ordered by unique_key so pages are stable, and sent with an app token to avoid throttling. After a successful run the watermark moves forward to the run time.

The upper bound on resolution_action_updated_date guards against bad dates. A check of the source on October 4, 2026 found one record, out of more than 22 million, with an update date in the future (December 14, 2026). It is a single outlier, but without a bound it would match the filter on every run, and in the MERGE a future date could make an old version look newer than a real later one. Silver therefore replaces a date later than the load time with the load time and sets a flag.

2.5 Continuity Between the Two Loads

The Full Load and the Incremental Load must not leave a gap, so that the tables stay live from the baseline onward. Three choices ensure this. First, the first incremental run looks back seven days, which overlaps the last six days of the baseline, so anything created or changed around T0 is picked up at least once. Second, the window start is the earlier of the last watermark and seven days ago, so a skipped run is recovered by the next one. Third, the overlap produces duplicate rows on purpose, and the Silver MERGE removes them by unique_key, so reruns are safe.

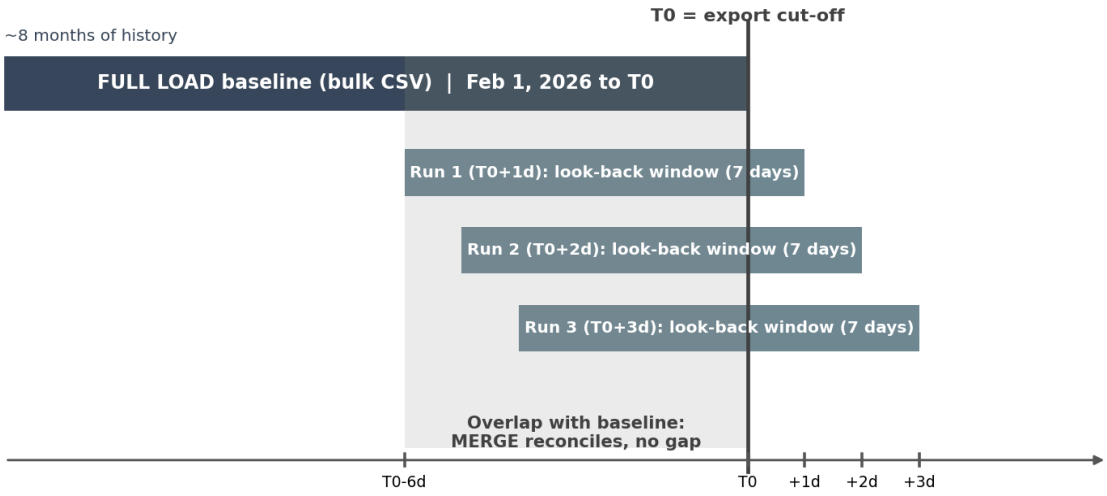


Figure 1. Full Load baseline and overlapping incremental windows

For example, request 61234501 is created on October 2 and is in the baseline with status Open. On October 4 at 08:00, after T0, the agency sets it to In Progress. The baseline never saw this change, but the first incremental run on October 5 looks back to September 28, finds the new resolution_action_updated_date, and the MERGE updates the Silver row.

Two checks run after every load. Freshness: the newest created_date in Silver must be within 48 hours of the run time, because the source runs about a day behind. Completeness: the Silver row count for the window is compared with a Socrata count(*) query using the same filter, and any difference is logged. Silver also keeps only records with created_date on or after the baseline start, so old tickets that happen to be updated recently do not enter the dataset.

When the source was checked on October 4, 2026, its newest created_date was October 3, 2026, 01:51, so the portal publishes about one day behind real time. In this project, live means the lakehouse is refreshed every day and trails the source by no more than the portal's own publishing delay.

2.6 Refresh of Open Tickets

A check on the source shows that resolution_action_updated_date is empty for many tickets that are still open. Among tickets created in the seven days before October 4, 2026, 8,355 of 11,927 In Progress tickets (70%) and 1,083 of 12,729 Open tickets (8.5%) had no value, while every Closed, Assigned and Pending ticket had one. A status change on such a ticket does not move the date, so once the ticket is older than seven days the incremental filter alone would not see the change. Closures are always captured because Closed tickets always have the date.

To close this gap, a lighter job runs once a week. It re-reads only four columns (unique_key, status, closed_date, resolution_action_updated_date) for tickets in scope that are not Closed and merges them into Silver. The row-update system field :updated_at was tested as an alternative, but it matched 562,817 records since October 3, 2026, against 2,079 for the date-based filter, so it is not selective enough to use.

2.7 Deleted Records

311 tickets are normally closed rather than deleted, so the dataset has no delete feed. If a key present in Silver is missing from a later full export during validation, it will be flagged in the completeness check rather than removed automatically.

3. Data Samples and Volume

3.1 Sample Files

Two sample files were downloaded from the source API with the same filters used by the pipeline and committed to the repository under sample_data/.

File	Format	Rows	Size	Filter used
full_load_sample.csv	CSV	5,000	3.81 MB	created_date Feb 1, 2026; first 5,000 rows by unique_key
incremental_sample.json	JSON	5,000	6.77 MB	7-day window from Sep 27, 2026; last 5,000 rows by unique_key

3.2 Volume and Frequency Estimate

NYC receives millions of 311 requests each year. A count query for the baseline range (February 1 to October 4, 2026) returned 2,653,620 rows. The 5,000-row Full Load sample is 3.81 MB, or about 0.76 MB per 1,000 rows, so the full baseline CSV is expected to be about 2.0 GB. That is far above the 200 MB requirement, and no duplication or synthetic data is needed. The dataset is wide (more than 40 columns, including long text and location fields), which is why the file is large.

Load	Format	Expected volume	Frequency
Full Load	CSV	About 2.0 GB (2,653,620 rows)	Once

Load	Format	Expected volume	Frequency
Incremental Load	JSON	About 110 to 130 MB fetched per run (80,923 to 96,683 rows), of which about 13 MB is newly created tickets	Daily

The Incremental Load estimate comes from two counts of the same seven-day window, 80,923 records and later 96,683 (the count grew as the portal published more data), and about 1.35 KB per record in the JSON sample. Around 9,900 new tickets are created per day (69,297 in seven days, roughly 13 MB), and the rest of the window is older tickets whose status changed. Because every run re-reads seven days, the daily download is larger than the net change, and the MERGE reduces it to the real inserts and updates.

3.3 Main Fields

Column	Raw type	Use
unique_key	string	Primary key and MERGE join key
created_date, closed_date, resolution_action_updated_date	string	Lifecycle timing and incremental filters
agency, agency_name	string	Responsible agency
complaint_type, descriptor	string	Request classification
status	string	Open, In Progress, or Closed
borough, incident_zip	string	Geographic aggregation
latitude, longitude	double	Coordinate validation
incident_address (house number), street_name, resolution_description	string	Possible personal information

4. Handling Updates with MERGE INTO

A 311 request is not a fixed event. It starts as Open, may move through Assigned, In Progress or Pending, and ends as Closed (Unspecified also appears in the data), and the source rewrites the same unique_key as its status changes. Appending every extract would leave several versions of one ticket in Silver, so the Silver layer uses Delta Lake MERGE INTO (an upsert) keyed on unique_key.

Batch	unique_key	status	resolution_action_updated_date
Already in Silver	61234501	Open	2026-10-02 09:10
Incoming, run 1	61234501	In Progress	2026-10-04 14:25
Incoming, run 4	61234501	Closed	2026-10-07 11:40

After run 4, Silver holds a single row for this ticket with status Closed. New keys are inserted and existing keys are updated.

```
-- keep only the latest version of each key in the batch
CREATE OR REPLACE TEMP VIEW staged_updates AS
SELECT * EXCEPT (rn) FROM (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY unique_key
    ORDER BY _ingest_ts DESC, resolution_action_updated_ts DESC) AS rn
  FROM cleansed_batch) WHERE rn = 1;

MERGE INTO silver.requests AS t
USING staged_updates AS s ON t.unique_key = s.unique_key
WHEN MATCHED AND s._ingest_ts > t._ingest_ts
  THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

The batch is de-duplicated first because Delta raises an error when more than one source row matches a target row, and the overlapping window often returns several versions of the same key. The extra condition on _ingest_ts stops an older record from overwriting a newer one if a batch is replayed. Versions are ordered by the time they were loaded into Bronze rather than by the source update date, because that date is empty for many open tickets and can be set in the future. Delta's transaction log (DESCRIBE HISTORY, VERSION AS OF) lets us show each status change after it is applied.

5. Security and Compliance

5.1 PII Analysis of the Samples

311 data is published by the city, but two fields hold text typed by residents or agency staff and can contain personal information: incident_address and resolution_description. Both sample files were scanned with a script (scripts/pii_scan.py in the repository) and 50 random rows of each text field were read by hand. Results:

Field	Check	Rows flagged	Of rows scanned
resolution_description	Phone-number pattern	124	4,984
resolution_description	Email pattern	10	4,984
resolution_description	Person names (read in full, non-template rows)	0	15
incident_address	Contains unit/apartment text	4	2,423
incident_address	House number present	1,799	2,423

Summary of findings: the 15 phone-like matches inspected were agency contact numbers (311, the health department, consumer protection and HPD), and all 10 email-like matches were the same agency inbox, so neither pattern pointed to resident details. About 88.5% of the 4,984 descriptions (4,412) follow eight standard agency templates, and none of the 15 non-template descriptions read in full contained a person's name. The sample covers one day of requests and free text can in principle hold resident details, so the field is still dropped. The address fields are more sensitive: where incident_address is present (2,423 of 5,000 rows) it usually holds a house number (1,799 rows) that identifies a building when joined with street_name. No direct identifiers such as account numbers or financial details appear in the structured columns.

5.2 Handling Strategy

Field	Risk	Action before Silver
incident_address	House number; with street_name it gives the exact address	Dropped. Only incident_zip and borough are kept
resolution_description	Free text with names or phone numbers	Dropped entirely
street_name, cross_street_1/2, intersection_street_1/2, landmark, location	Rebuild the address	Dropped
latitude, longitude	Precise point can identify a household	Used to validate coordinates, then rounded or dropped before Gold

Raw values exist only in Bronze, which is not used by the dashboard. Silver and Gold contain no direct identifiers, and no personal field is copied into a dimension table.

6. Medallion Data Model

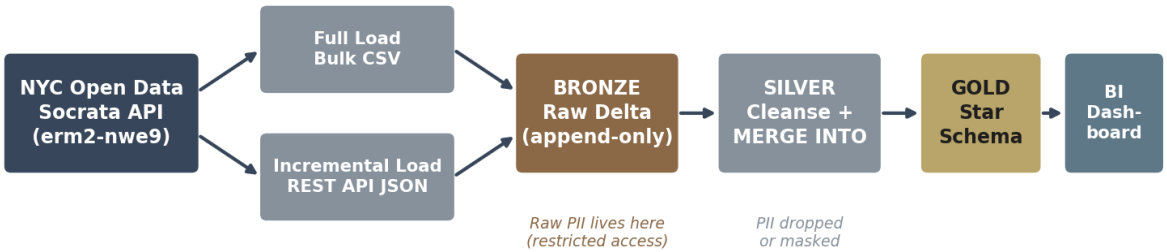


Figure 2. Pipeline overview

6.1 Bronze

Bronze stores the data as received. The CSV (Full Load) and JSON (Incremental Load) are written to Delta tables with every column kept as a string, so a change in the source format does not break ingestion. Three audit columns are added: _ingest_ts, _source_file, and _load_type (FULL or INCR). No cleaning happens here, so Bronze can be replayed if a later step has a bug.

6.2 Silver

Moving from Bronze to Silver involves the following steps.

1. Drop or mask the personal fields listed in Section 5.2.
2. Cast created_date, closed_date and resolution_action_updated_date from strings to timestamps.
3. Replace a resolution_action_updated_date later than the load time with the load time and set the is_future_dated flag.
4. Remove rows with invalid coordinates: null, zero, or outside the NYC bounding box (about latitude 40.4 to 41.0 and longitude -74.3 to -73.6).
5. Standardize text values in borough, status and agency (trim, upper-case, handle nulls). The six status values (Open, Assigned, In Progress, Pending, Unspecified, Closed) are kept as received.

6. Keep one row per unique_key and apply the MERGE INTO from Section 4.

Planned Silver table silver.requests:

Column	Type	Note
unique_key	STRING	Primary key
created_ts	TIMESTAMP	Cast from created_date
closed_ts	TIMESTAMP	Null while the ticket is open
resolution_action_updated_ts	TIMESTAMP	Used by MERGE and incremental filter
agency, agency_name	STRING	Standardized
complaint_type, descriptor	STRING	Standardized
status	STRING	Open, In Progress, Closed
borough, incident_zip	STRING	Only location attributes kept
latitude, longitude	DOUBLE	Rounded to 3 decimals
is_future_dated	BOOLEAN	True if the source update date was after the load time
_ingest_ts, _load_type	TIMESTAMP, STRING	Lineage columns from Bronze; _ingest_ts orders versions in the MERGE

Some tickets with status Assigned or In Progress already have a closed_date, so Silver keeps both fields as received and calculates resolution_hours and is_resolved only when the status is Closed.

6.3 Gold

Gold is a star schema built from Silver. The fact table has one row per service request, and four dimensions describe where, who, what and when.

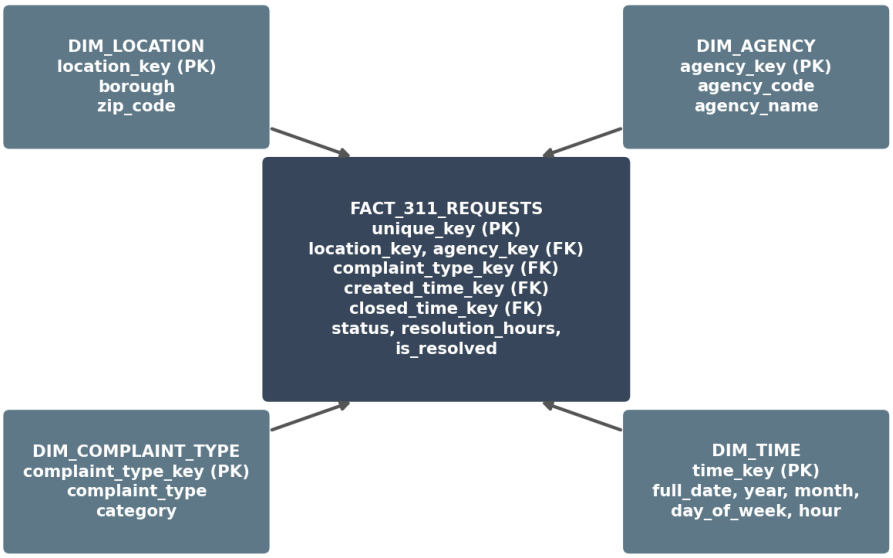


Figure 3. Gold star schema

Table	Contents
Fact_311_Requests	One row per request; keys to each dimension, status, resolution_hours, is_resolved
Dim_Location	Borough and zip code (no street-level data)
Dim_Agency	NYPD, DSNY, DOT, HPD, DEP and others
Dim_Complaint_Type	Noise, Rodent, Illegal Parking and others
Dim_Time	Date and hour attributes, used for both created and closed dates
agg_daily_requests	Pre-aggregated counts and average resolution hours by day, borough, agency and complaint type, for fast dashboard queries

7. Business Intelligence Dashboard

The dashboard will be built with Databricks dashboards on top of the Gold tables. It will answer three questions.

#	Metric	Chart	Question answered
1	Average resolution time by agency	Horizontal bar chart	Which agencies close requests fastest and slowest?
2	Rodent complaints by zip code	Heatmap or map	Where are rodent problems concentrated?
3	Top 5 unresolved complaint types	Ranked bar chart	Which kinds of complaint have the biggest open backlog?

```
SELECT a.agency_name,
       ROUND(AVG(f.resolution_hours), 1) AS avg_resolution_hours
FROM gold.fact_311_requests f
JOIN gold.dim_agency a ON f.agency_key = a.agency_key
WHERE f.is_resolved = TRUE
GROUP BY a.agency_name
ORDER BY avg_resolution_hours DESC;
```

8. Engineering Setup and FinOps

8.1 Version Control

https://github.com/talmeez450/nyc311-medallion-lakehouse). Both sample files are committed under sample_data/. Only small slices are committed because GitHub rejects files over 100 MB; the full baseline stays out of the repository. Planned layout:

```
nyc311-medallion-lakehouse/
  README.md
  sample_data/     full_load_sample.csv, incremental_sample.json
  scripts/         get_samples.py, pii_scan.py
  notebooks/       bronze, silver, gold (added in Phase 2)
  docs/            Phase 1 proposal
```

8.2 Platform

The pipeline will run on Databricks Free Edition, which replaced the older Community Edition. Free Edition uses serverless compute only, supports Python and SQL, includes Delta Lake and dashboards, and has a fair-use quota: when the quota is exceeded, compute is unavailable for the rest of the day. Free Edition also limits a workspace to 500 tables.

8.3 Cost and Quota Management

- Development and testing use a one-week slice of the data. The eight-month Full Load is run once, after the notebooks are working.
- Bronze is never re-ingested unless the source file changes. Silver and Gold are rebuilt from Bronze instead of downloading again.
- Notebooks are short batch jobs with no always-on streaming, so compute is used only while a run is active.
- Raw files are stored once in a Unity Catalog volume. Gold tables are small because they are aggregated, and OPTIMIZE is run only after large loads.
- The pipeline uses a small fixed set of tables (three layers plus the star schema and a control table), well under the 500-table limit.
- Work is spread across days if the daily quota is reached, and the incremental watermark makes it safe to resume after an interruption.

9. Risks

Risk	Mitigation
Gap between Full Load and Incremental Load	Recorded cut-off T0, overlapping first window, watermark recovery, count check against Socrata
API throttling or pagination errors	App token, ordered pages, retry with backoff
Status changes missed on older open tickets	Weekly narrow status refresh of tickets that are not Closed; closures are always captured by the main filter
Duplicate keys breaking MERGE	ROW_NUMBER de-duplication before MERGE
Free Edition quota exhausted	Small test slices, run the Full Load once, resume from the watermark
Personal data leaking into later layers	Drop at the Silver boundary, restrict Bronze, rerun the PII scan on Silver output
Source schema changes	Bronze keeps raw strings; types applied only in Silver